

Resident OS — Build Steps

File-by-file, in the order you create them. Companion to the 3-page architecture poster and the Claude Code task kit.

Each step is one file (or one small group). Build them in order — every step depends only on things above it. Steps marked **[HUMAN]** you write yourself; delegating them defeats the purpose.

Phase 0 — Foundations and contracts

- 1. requirements.txt / pyproject.toml** — Pin every version. LangGraph's API has moved between releases, so an unpinned install will break you. Core set: `fastapi`, `uvicorn`, `pydantic`, `sqlalchemy`, `asyncpg`, `alembic`, `httpx`, `redis`, `anthropic`, `openai`, `langgraph`, `langchain-core`, `sentence-transformers`, `presidio-analyzer`, `pytest`, `pytest-asyncio`, `ruff`, `mypy`.
- 2. .env / .env.example** — API keys (Anthropic, OpenAI for embeddings, Google for the judge), `DATABASE_URL`, `REDIS_URL`, R2 credentials, Supabase keys, plus behaviour flags: `DEMO_MODE`, `COUNCIL_THRESHOLD`, `CONFIDENCE_AUTO_THRESHOLD`, `CONFIDENCE_ABSTAIN_THRESHOLD`, `MAX_GRAPH_STEPS`, `ORG_DAILY_BUDGET_USD`. Commit `.env.example` only. Confirm `.env` is in `.gitignore` before your first commit.
- 3. CLAUDE.md** — The invariants your AI assistant reads before every session. Fourteen rules, a no-scaffolding clause, and the prompt-injection posture. This is what stops two months of sessions producing four architectures.
- 4. infra/docker-compose.dev.yml** — Postgres (`pgvector/pgvector:pg16`) and Redis. Nothing else yet. Add services when the code that needs them exists.
- 5. infra/migrations/0001_init.sql** — The complete schema, written once: domain tables (orgs → properties → buildings → units, people, roles, tenancies, assets, tickets, work_orders, vendors), the AI spine (agent_runs, agent_steps, llm_calls, retrievals, guardrail_events, decisions, approvals), knowledge and memory (kb_documents, kb_chunks, episodic_memory, reflections), and evaluation (eval_datasets, eval_runs, failure_events, prompt_versions, metric_rollups, label_queue). **No column anywhere for race, ethnicity, religion, disability, familial status, national origin, immigration status, orientation, or health.** Not nullable, not unused — the columns must not exist.
- 6. infra/migrations/0002_rls.sql** — Row Level Security on every table, the tenant-isolation policy, the resident-own-unit policy, and the append-only triggers on `decisions`, `llm_calls`, and `guardrail_events`.
- 7. docs/vocabularies.md** — Exact enum strings for priority, ticket_status, responsible_party, trade, reject_reason, decision_mode, guardrail, notification_tier, role, authority, failure_kind. Looks trivial; it is the most common source of late integration pain.
- 8. docs/openapi.yaml** — Hand-written **before** any endpoint exists. Every path, request schema, response schema, and error code. Your frontend and your assistant both build against this instead of inventing shapes.

9. docs/events.md — The SSE contract: `step.started`, `step.finished`, `retrieval.done`, `council.opened`, `council.member`, `token`, `decision.ready`, `guardrail.hit`, `run.failed`, with exact payload fields.

10. services/brain/src/brain/contracts.py — Strict Pydantic models shared by every agent: `TicketFacts`, `ContextEnvelope`, `Claim`, `Diagnosis`, `DispatchPlan`, `AuditResult`, `MessageDraft`, `JudgeScores`, `TicketState`, `SafetyVerdict`. Agents never exchange free text — every handoff is one of these.

Phase 1 — Running skeleton

11. infra/seed/generate.py — Deterministic synthetic data from a fixed seed: 1 org, 2 properties (US-VA and US-MD), 400 units, 320 tenancies, 40 assets, 8 vendors, 1,200 historical tickets with realistic category and timing distributions. **Unit 4B gets exactly 3 kitchen-drain tickets over 14 months** and **unit 2C gets a 2019 water heater under warranty to 2027-03** — every later demo uses these two.

12. services/api/src/api/db.py — Async engine, session factory, and the per-request RLS context setter. Never construct a service-role client on a user path.

13. services/api/src/api/deps/auth.py — Verify the Supabase JWT, extract `sub`, `org_id`, `person_id`, `role`.

14. services/api/src/api/deps/tenancy.py — Set the RLS context on the connection for the request. Belt and suspenders with explicit `org_id` scoping in every query.

15. services/api/src/api/middleware/idempotency.py — Required on every POST, PUT, PATCH. Cache the response by `Idempotency-Key` for 24h. Agentic systems retry; without this, a retry becomes a duplicate real-world action — two work orders, two vendor dispatches, two texts to a resident.

16. services/api/src/api/middleware/errors.py — RFC 7807 problem details with stable `type` URIs. Never return a raw exception string to a client.

17. services/api/src/api/main.py — Assemble the pipeline in order: `auth` → `tenancy` → `rate limit` → `idempotency` → `routes` → `errors`. Health check reporting DB and Redis connectivity.

18. services/api/tests/test_tenancy.py — Parametrised over **every** registered route: a token scoped to org A requesting an org-B resource must never return 200. Write this now, not later — retrofitting it after twenty endpoints means auditing twenty endpoints. It runs in CI forever.

19. services/api/src/api/routers/tickets.py — `GET /v1/tickets` (cursor pagination on `(created_at, id)`, never offset) and `GET /v1/tickets/{id}`.

20. apps/web/ — Next.js 15 App Router scaffold with five route groups: `(resident)/app`, `(manager)/manage`, `(owner)/owner`, `(tech)/tech`, `v/[token]`. Auth flows: magic link for residents, password + TOTP for staff, signed HMAC links for vendors (no account).

21. packages/shared-types/ — TypeScript types **generated** from `docs/openapi.yaml` via a `pnpm generate` script. Never hand-write a type that mirrors a backend model; that is how definitions drift.

22. packages/ui/ — Three signature components, shadcn defaults for everything else: the **status lamp** (priority, always paired with a text label so colour never carries meaning alone), the **evidence**

rail (citation chips with two-way hover), and the **machine-blue** / **human-brass** semantic — blue means the system decided, brass means a person authorised.

Phase 2 — Knowledge and retrieval

23. knowledge//* .md [HUMAN]** — Twelve real documents: plumbing, HVAC, electrical and appliance SOPs, an emergency procedure, a lease template, two jurisdiction habitability SLA tables, community rules, vendor procedures, resident FAQ. Every file needs front matter with `id`, `version`, `effective_from`, `effective_to`, `jurisdiction`, and `authority`. Include verbatim: **SOP-PLM-04** ("after two failed drain-clearing attempts within 12 months, replace the P-trap assembly") and **lease §7.3** (owner pays for normal wear and tear). Generated documents will be generically plausible and every downstream eval will be measuring against fiction.

24. services/brain/src/brain/retrieval/chunking.py — A strategy per corpus: leases clause-level keeping the section path, SOPs heading-aware at 300–600 tokens with 15% overlap, community rules one per chunk, work-order history one chunk per ticket embedding `symptom + resolution`.

25. services/brain/src/brain/retrieval/embed.py — `text-embedding-3-small`, stored as `halfvec(1536)` to halve index memory, cached by `sha256(text) + model`.

26. services/brain/src/brain/retrieval/ingest.py — Parse front matter, validate `authority` against the enum and **fail loudly** on a bad value (a silent default corrupts precedence forever), chunk, embed, upsert. Skip any document whose `content_sha256` is unchanged.

27. evals/datasets/retrieval_v1.jsonl [HUMAN] — 50 query → expected-document pairs, written **before** the retriever exists so you don't unconsciously write queries it already handles. Six types: natural resident phrasing, identifier lookups ("SOP-PLM-04", "lease section 7.3"), synonym/paraphrase, jurisdiction-specific, effective-date sensitive, and five that should return nothing.

28. services/brain/src/brain/retrieval/hybrid.py — BM25 over `tsvector` (GIN) and dense over `pgvector` (HNSW), each top 30, fused with reciprocal rank fusion at `1/(60 + rank)`. Metadata filters applied **after** fusion — pre-filtering a selective predicate collapses HNSW recall. Effective-date filter uses the ticket's timestamp, not `now()`.

29. services/brain/src/brain/retrieval/rerank.py — `bge-reranker-v2-m3` cross-encoder, 30 → 6. Load the model **once at module level**; per-request loading would dominate the latency budget. Persist `rerank_score` into `retrievals.results`.

30. services/brain/src/brain/retrieval/parent.py — Expand each retrieved chunk to its containing section. A policy clause is meaningless without its surrounding conditions.

31. services/brain/src/brain/context/compress.py — **Extractive** sentence selection against the query. Never abstractive: summarising policy text introduces errors upstream of the model, on exactly the text that carries legal weight.

32. services/brain/src/brain/context/envelope.py — Assemble context with provenance IDs (`[C1]`, `[F1]`) carrying source, version, effective date, and section. The model is instructed that every factual claim must cite an ID or move to `unknowns`.

33. services/brain/src/brain/context/budget.py — Per-slot token caps totalling 8,000, enforced with a real tokenizer: system+schema 1,000 (static, so it caches), policy 1,600, unit/asset facts 700 (key-

value, never JSON dumps), similar cases 1,300, conversation summary 500, current ticket 600 (never truncated), output reserve 2,300.

34. `evals/retrieval.py` — Measure recall@5/10, MRR, nDCG@10 across four strategies: dense only, lexical only, hybrid RRF, hybrid + rerank. **Publish the table with the git SHA.** If reranking doesn't help, delete it.

35. `apps/web/src/app/(resident)/app/report/` — Three-step submission: six large category tiles (no dropdowns), camera-first photo sheet, access permission and preferred window. **The acknowledgment must return in under a second and must never wait on a model call** — enqueue, then respond.

Phase 3 — The agent loop

36. `services/brain/src/brain/gateway/client.py` — The single function every model call goes through: `complete(task, prompt, schema)`. Routing table mapping task class to tier (small/mid/large), prompt loading with content hashing, prompt caching on the static prefix, timeout, two retries with jitter, circuit breaker per provider, and a `llm_calls` row written on every call. **No Anthropic SDK call anywhere else in the codebase.**

37. `services/brain/src/brain/prompts/*.md.j2` — Every prompt as a versioned file, content-hashed at load, hash recorded on each call. Never inline a prompt string in Python.

38. `services/brain/src/brain/agents/safety.py` — Regex patterns for gas, fire, flood, electrical, CO, no-heat and sewage, plus a small-model second opinion. **OR logic, never AND** — either signal triggers. Tune for recall (target 0.99), accept false positives; missing a gas leak is unbounded, a false alarm costs one phone call.

39. `services/brain/src/brain/agents/p0_protocol.py` — Page on-call via push, SMS and voice simultaneously, quiet hours ignored, fixed templates. **Zero model calls in this file.** Nothing probabilistic between detection and the phone.

40. `evals/datasets/safety_v1.jsonl` [HUMAN] — 40 items: 20 true emergencies and 20 near-misses ("smells like garbage", "the stove smells when I cook", "small drip under the sink"). The near-misses are what tune precision without costing recall.

41. `services/brain/src/brain/agents/intake.py` — Free text plus photo captions into `TicketFacts`. Strict schema, one repair retry with the validation error fed back, then a human. Include an `intent` field (`maintenance` | `question` | `other`) — one field now saves a whole agent later.

42. `services/brain/src/brain/context/broker.py` — Fetch policy, unit history, asset record and episodic memory with `asyncio.gather`. They are independent; a sequential await chain here is the most commonly missed latency win. Persist all five ranking fields per chunk (`bm25_rank`, `vec_rank`, `rrf`, `rerank_score`, `used`) — without them the retrieval inspector cannot be built later without re-running history.

43. `services/brain/src/brain/agents/diagnostician.py` — Grounded cause and recommended actions. A plain async function with no tools and no LangGraph import; context is pushed to it, not pulled.

- 44. `services/brain/src/brain/guardrails/citations.py`** — Deterministic verification: every cited ID exists in the envelope, every sentence containing a number, date, dollar amount or \$ carries a citation, and every cited figure appears verbatim in the referenced chunk. One targeted regeneration naming the specific claims, then a human. **The highest-value anti-hallucination component in the project, and it costs nothing.**
- 45. `services/brain/src/brain/policy/precedence.py`** — Conflict resolution in code: `statute > lease > internal_sop > vendor_contract`.
- 46. `services/brain/src/brain/agents/dispatch.py`** — Trade, in-house vs vendor, vendor selection scored on accept rate and first-time-fix, window, parts, cost estimate. Never propose a vendor with expired insurance or outside their service area. **Proposes only — never writes.**
- 47. `services/brain/src/brain/agents/auditor.py`** — Check the decision against lease, statute and SOP. It sees the decision output and the policy and **deliberately not the diagnostic reasoning chain** — a model that reviews its own reasoning agrees with itself far more than the reasoning deserves.
- 48. `services/brain/src/brain/graph/state.py`** — The shared `TicketState` carrying facts, envelope, diagnosis, plan, audit result, decision, confidence, retries and trace.
- 49. `services/brain/src/brain/graph/nodes.py`** — Three-line wrappers around the plain agent functions. This is the only place `LangGraph` is imported outside the builder, and it is what keeps the core framework-agnostic.
- 50. `services/brain/src/brain/graph/builder.py`** — Assemble the graph with conditional edges (safety → P0 or context; gate → auto / approve / escalate), `PostgresSaver` checkpointing so a recycled container resumes rather than restarts, `interrupt_before=["approval"]`, and a hard 12-node step cap.
- 51. `services/brain/src/brain/guardrails/tool_auth.py`** — Server-side grant map checked on every tool call. Side-effecting tools additionally require a single-use, action-scoped approval token. A prompt saying "you may only read" is a suggestion; this is a control.
- 52. `services/brain/src/brain/degraded.py`** — Rules-only mode in roughly 150 lines: keyword safety screen, category default SLA, fixed acknowledgment, everything marked escalated. With zero model providers reachable, a resident still gets a ticket number and a gas leak still pages the on-call.
- 53. `services/api/src/api/routers/stream.py`** — SSE emitting the `docs/events.md` contract. The step timeline renders immediately with pending steps hollow — structure arrives before content, and it is the most persuasive eight seconds in the product.
- 54. `services/api/src/api/routers/approvals.py`** — The queue **ordered by SLA risk, never arrival time**, plus approve, reject (with a five-option reason taxonomy that becomes your eval labels), and reassign.
- 55. `apps/web/src/app/(manager)/manage/`** — The approval queue: one card, one screen, no scrolling. Keyboard-first (`j/k`, `a`, `e`, `r` then `1–5`, `t`). The decider diamond flips from blue to brass the instant a human touches it. Any claim without a citation chip renders in a visibly "unsupported" style — if the system produces one, the interface must not hide it.
- 56. `services/api/Dockerfile` + `gcloud run deploy`** — Cloud Run with `min-instances=1` on the API only (cold start hits the exact request a reviewer makes, and the cross-encoder must stay resident); worker, MCP and observability scale to zero. Frontend to Vercel. **Set `DEMO_MODE=true` in production** until you have verified every outbound channel is blocked.

Phase 4 — Guardrails

57. `services/brain/src/brain/guardrails/shield.py` — A `ShieldProvider` protocol with `LocalShield` as the always-on implementation. Managed providers augment it later, combine pessimistically (any block is a block), and time out at 250ms into a logged degradation rather than a failed request. **A classifier is a filter; the architecture is the boundary.**

58. `services/brain/src/brain/guardrails/pii.py` — Presidio plus regex for SSN, card, phone, email, DOB. Redact before any prompt is assembled. `agent_steps` stores an `input_digest` (sha256), never the payload.

59. `services/brain/src/brain/guardrails/injection.py` — Trust levels on every string entering context, untrusted content wrapped in labelled delimiters, and a pattern scanner as the second line. The real control is that untrusted content never reaches an agent holding write tools.

60. `evals/datasets/injection_v1.jsonl` — 40 attempts across **every** entry point, not just chat: resident text, vendor SMS reply bodies, PDF text layers, image captions, marketplace listings. Almost everyone scans the chat box and forgets the rest.

61. `services/brain/src/brain/guardrails/fair_housing.py` — Term blocklist on output, intent classifier over the drafted message, and a block that routes to a human — **never a silent rewrite.**

62. `evals/datasets/redteam_fh.jsonl` — Paired probes identical except for one protected-class signal (voucher holder, wheelchair access, service animal, family with children, non-English name), asserting response **parity**: same outcome, similar completeness, same latency class. This is exactly the test a fair-housing testing organisation would run against you.

63. `apps/web/src/components/GuardrailBlock.tsx` — What was blocked, why, and what the human must do next.

Phase 5 — Evaluation and ML ops

64. `evals/datasets/golden_v1.jsonl` [HUMAN — do not delegate] — 80 labelled tickets, deliberately skewed toward hard cases: ambiguous priority, chargeback disputes, life-safety near-misses, recurring failures, multilingual, and four where the correct answer is to **ask**. Each item carries `must_cite` document IDs and `acceptable_alternatives`, because several genuinely have two defensible answers. **Label these before you tune any prompt** — labelling afterwards encodes the model's current behaviour as ground truth.

65. `evals/metrics.py` — Priority macro-F1, party accuracy, trade accuracy, P0 recall and precision, groundedness (deterministic, from the citation verifier — free), escalation precision and recall, cost per ticket, p50/p95 latency.

66. `evals/run.py` — Run every item through the full workflow and write results to `eval_runs` with `git_sha`, `judge_model`, `judge_version` and `seed`.

67. `services/brain/src/brain/judge/grader.py` — Offline 100% in CI, online 10% sample, always async and never blocking. **Route to a different model family than you generate with** to avoid self-preference bias, normalise for length, and pin the version — a silent judge upgrade invalidates your entire time series.

- 68. evals/bands.yaml + evals/gate.py** — Tolerance bands rather than exact thresholds, with a seeded sample. Model nondeterminism makes exact gates flaky, a flaky gate gets ignored, and an ignored gate is worse than no gate. `p0_recall` gets zero regression tolerance; `fair_housing` gets `max_violations: 0`.
- 69. .github/workflows/evals.yaml** — The eval gate that blocks merges. Verify it actually fails by deliberately breaking a prompt on a throwaway branch.
- 70. services/brain/src/brain/confidence/blend.py** — Blend retrieval support, member agreement, discounted self-report, historical category accuracy and schema cleanliness. Models are badly calibrated when simply asked how confident they are.
- 71. services/brain/src/brain/confidence/calibrate.py** — Isotonic regression fit on golden-set deciles, plus expected calibration error and the reliability diagram. Almost no portfolio project has a calibration curve.
- 72. services/api/src/api/routers/runs.py** — GET `/v1/runs/{id}` returning the full nested trace, and GET `/v1/runs/{id}/replay` re-executing against pinned prompt, model and KB versions with a diff. Replay is how you debug, how you prove non-regression, and how you answer an auditor.
- 73. apps/web/src/components/TraceTree.tsx** — Virtualised nested span tree with parallel spans sharing a horizontal track. **Label steps with no model call as code** — this makes the ≤ 5 -call budget visible.
- 74. apps/web/src/components/RetrievalInspector.tsx** — BM25, vector, fused and reranked columns with a **budget cut line**. Chunks below it are dimmed but present; what was *almost* retrieved is usually the answer to "why was this wrong".
- 75. infra/migrations/0003_ops.sql** — `failure_events`, `prompt_versions`, `metric_rollups`, `label_queue`. `label_queue.expected` is nullable **on purpose** — a production failure enters an eval dataset only after a human writes the expected answer.
- 76. services/worker/src/worker/rollups.py** — Hourly metric rollups. Every ops panel reads these; **no dashboard queries a raw event table**, because observability that gets heavier exactly as things degrade is worse than none.
- 77. services/api/src/api/routers/ops.py** — `/v1/ops/health`, `/agents`, `/failures`, `/prompts`, `/drift`, `/outputs`. **Admin role only** — it exposes prompt hashes, raw outputs and cost internals. CI asserts a manager token gets 403.
- 78. apps/web/src/app/ops/** — One shell with a single global time range, not four routes. Panels: health verdict and quadrants, agent scorecard **ranked by degradation rather than alphabetically** with a rule-generated change narrative (a model writing it would occasionally invent a cause), the failure feed with promote-to-label-queue, and the eval history with the calibration diagram.
- 79. docs/evals/REPORT.md [HUMAN]** — Every real number with its git SHA, **including the ones you missed**. An eval report with three red numbers and honest explanations is more credible than five green ones, and it is the fastest signal that you have actually operated something.
- 80. README.md + apps/web/src/app/page.tsx [HUMAN]** — Architecture diagram, a 90-second silent GIF of the trace streaming, four badges with real values, and three buttons dropping straight into pre-logged-in demos. No form, no gate. Note the synthetic dataset on the page — "400 units, 1,200 historical tickets" reads as rigour, not as a caveat.
-

The rule that governs all eighty steps

No number ships that you cannot reproduce from a committed eval run and a git SHA. Not in the README, not in a blog post, not on a resume. If you don't have a measured value yet, write [TBD from eval run] and move on.